PERTEMUAN 3: PERULANGAN

3.1. Tujuan

1. Mahasiswa memahami konsep perulangan pada C++.
2. Mahasiswa mampu mengimplementasikan perulangan `for`, `while`, dan `do-while`.

3.2. Dasar Teori

Tidak jarang ketika membuat program dibutuhkan lebih dari sekali eksekusi untuk perintah atau baris kode tertentu. Melakukan perintah secara berulang pada baris kode tertentu dinamakan perulangan (iterasi). Implementasi perintah perulangan tidak dilakukan dengan cara hanya menduplikasi perintah kode tetapi menggunakan konsep perulangan yang telah disediakan oleh bahasa pemrograman C++ yaitu perulangan menggunakan `for`, `while`, dan `do-while`. Pada pertemuan kedua ini akan dipelajari konsep dan implementasi perulangan di dalam C++.

3.2.1. Perulangan `for`

Perulangan for, merupakan sebuah struktur kendali yang digunakan untuk mengeksekusi blok kode secara berulang dalam jumlah iterasi yang ditentukan. Perulangan menggunakan `for` memungkinkan baris kode program untuk dieksekusi dalam rentang urutan angka tertentu. Misalkan, ketika ingin menampilkan setiap karakter dari sebuah variabel dengan tipe data string. Sintaks perulangan menggunakan `for` secara umum adalah sebagai berikut,

```
for (int i = batas_awal; i < batas_akhir; i++)
      statement;
```

Makna dari sintaks for di atas adalah lakukan perulangan pada statement sebanyak `batas_akhir-batas_awal` dan lakukan secara inkremental `i++`. Ekspresi `i++` dapat dituliskan sebagai `i = i + 1`. Seperti halnya pada percabangan `if`, ketika terdapat lebih dari satu `statement` maka diperlukan penulisan beberapa `statement` tersebut di dalam kurung kurawal.

```
for (int i = batas_awal; i < batas_akhir; i++){
    statement;
    statement;
}
```

Potongan kode di bawah ini adalah contoh implementasi penggunaan perulangan `for`,

```
for (int i = 0; i < 5; i++)
    cout << i << “\t” << i*i << endl;
```

Potongan kode di atas akan menampilkan nilai `i` dan kuadratnya secara berulangan dimulai dari `i = 0` hingga `i = 5` secara inkremental. Sehingga hasil yang diperoleh adalah sebagai berikut,

```
0   0
1   1
2   4
3   9
4   16
```

Selain secara inkremental, perulangan juga dapat dilakukan secara dekremental,

```
for (int i = 5; i > 0; i--)
    cout << i << “\t” << i*i << endl;
```

Perhatikan bahwa nilai `i` dimulai dari `i = 5` dan `i > 0`. Ekspresi dekremental dituliskan sebagai `i--` atau dapat juga dituliskan sebagai `i = i – 1`. Hasil yang diperoleh adalah

```
5   25
4   16
3   9
2   4
1   1
```

3.2.2. Perulangan `while`

Merupakan sebuah struktur kendali yang mengeksekusi blok kode selama kondisi bernilai benar (True). Perulangan `while` adalah teknik perulangan lain yang disediakan oleh bahasa C++ selain perulangan `for`. Secara umum pengulangan `while` dituliskan sebagai berikut,

```
while (kondisi){
    statement;
}
```

Contoh implementasi penggunaan perulangan `while` adalah potongan kode berikut ini,

```
int x = 1;
while (x <= 5){
    cout << “x: “ << endl;
    x++;
}
```

Potongan kode diatas akan terus mencetak nilai `x` selama nilai `x` memenuhi kondisi `x <= 5`. Kondisi ini terpenuhi dikarenakan setiap pengulangan nilai `x` mengalami inkremen (nilai bertambah) melalui statement `x++`. Apa yang akan terjadi ketika `x++` dihapus dari potongan kode di atas?

3.2.3. Perulangan Bersarang (*Nested Loop*)

Perulangan bersarang adalah perulangan yang terdapat di dalam perulangan. Perhatikan potongan kode di bawah ini yang merupakan implementasi dari perulangan bersarang menggunakan `for`,

```
for (int i = 0; i < 4; i++){
    for (int j = 0; j < 4; j++){
          cout << “X” << “\t”;
    }
    cout << endl;
}
```

Hasil yang diperoleh pada potongan kode di atas adalah

```
X   X     X     X
X   X     X     X
X   X     X     X
X   X     X     X
```

Perulangan bersarang juga dapat dilakukan untuk perulangan `while` dan kombinasi dari `for-while`. Tidak ada batasan jumlah perulangan di dalam pengulangan, namun semakin banyak perulangan akan berpengaruh kepada beban komputasi yang dibutuhkan untuk mengeksekusi program.

3.2.4. Perulangan `do-while`

Perulangan do-while adalah perulangan yang mengeksekusi statement secara berulang selama kondisi pada perulangan while terpenuhi atau benar. Sintaks umum perulangan do-while adalah

```
do {
    statement;
} while (kondisi);
```

Contoh implementasi perulangan do-while ditunjukan pada potongan kode di bawah ini,

```
int tebakan;
int angka = 5;

do {
    cout << “Tebak angka 1-10: “;
    cin >> tebakan;
} while (tebakan != angka);

cout << “Tebakan Anda benar!” << endl;
```

Blok kode di dalam `do{}` akan terus berulang selama `tebakan != angka` benar. Diketahui bahwa `angka = 5`, jika `tebakan = 5` maka `5 != 5` sehingga perulangan pada blok kode `do{}` dihentikan. Hasil eksekusi diperoleh,

```
Tebak angka 1-10: 2
Tebak angka 1-10: 3
Tebak angka 1-10: 1
Tebak angka 1-10: 9
Tebak angka 1-10: 4
Tebak angka 1-10: 5
Tebakan anda benar!
```

3.2.5. *Infinite Loop*

Dalam menerapkan konsep perulangan terdapat suatu permasalahan yang akan sering ditemukan yaitu *infinite loop*. *Infinite loop* merupakan suatu kondisi dimana *loop* akan terus berjalan secara berulang-ulang tanpa henti. Dalam kata lain infinite loop merupakan perulangan yang kondisinya akan selalu true sehingga loop akan terus mengeksekusi kode yang ada didalamnya.

Efek yang ditimbulkan pada program yaitu akan membuat program menjadi *unresponsive* . Ini bisa terjadi karena sebuah loop tidak memiliki ***false condition*** yang membuat mereka berhenti sehingga *loop* akan terus berjalan.

```
for (int i=0; i >= 0; i++) {
    cout << "Kok gak berhenti??" << endl;
}
```

Cara mengatasi *infinite loop* kita dapat mengubah kondisi atau *decrement/increment* nya.

```
for (int i=0; i <= 5; i++) {
    cout << "Loop berhenti di putaran ke " << i << endl;
}
```

Hal ini juga berlaku sama ke perulangan `while()` ataupun `do while()`.

Perlu diketahui, terkadang perulangan `while()` ataupun `do while()` sering beresiko terjadi *infinite loop* karena lupa untuk memasukkan kondisi.

```
int x = 5;

while (x >= 0) {
    cout << "Loop berhenti di putaran ke " << i << endl;
    x--;
}
```

```
int x = 5;

do {
    cout << "Loop berhenti di putaran ke " << i << endl;
    x--;
} while (x >= 0);
```

3.2.6. *Control Flow*

Dalam perulangan terdapat beberapa *control flow* yang bisa digunakan terutama dalam bahasa C++ seperti **Continue**, **Break**. *Control flow* dapat membuat kita memanipulasi *loop* yang terjadi dan membuat output menjadi sesuai yang kita inginkan. `Continue` pada *loop* seperti sebuah tombol *"skip"* yang biasa ditemukan di *playlist youtube*. Jika diterapkan pada loop `for()` dan `while()` bisa digunakan untuk lompat ke iterasi selanjutnya sehingga bila terdapat sebuah kondisi yang tidak diinginkan maka dengan `continue` kita bisa loncat ke iterasi berikutnya. Apabila kita ingin menampilkan angka genap dari 1-10 maka kita bisa menggunakan `continue`.

```
int main(){
    for (int i = 1; i <= 10; i++){
          if (i % 2 != 0) {
                continue; //skip nomor ganjil
          }

          cout << i << " "; // 2 4 6 8 10
    }

    return 0;
}
```

`Break` seperti namanya yaitu seperti tanda *stop*. Ketika kita menggunakan `break` dalam *loop* maka otomatis semua *loop* nya akan berhenti dan selesai. Break dapat digunakan jika kita sudah menemukan kondisi yang diinginkan sehingga seluruh loop akan berhenti. Contoh:

```
int main() {
    string mahasiswa[] = {"Andi", "Budi", "Charlie", "David", "Edi", "Fajar"};

    for (int i = 0; i < 5; i++){
          if (mahasiswa[i] == "Charlie"){
                cout << mahasiswa[i] " ditemukan dalam data mahasiswa" << endl;
                break; //keluar loop ketika kondisi yang diinginkan sudah ketemu
          }
    }

    return 0;
}
```

3.3. Aktivitas Praktikum

1. Buatlah program yang dimana kita memasukkan sebuah angka (misalnya 100). Program harus menghitung berapa kali angka tersebut bisa dibagi 2 secara terus-menerus hingga hasilnya menjadi kurang dari 1. Tampilkan hasil pembagian dari tiap perulangan.
2. Buatlah program yang dimana kita memasukkan satu bilangan bulat, lalu hitung dan tampilkan hasil faktorialnya.

3. Buatlah sebuah program yang dimana kita memasukkan sebuah angka batas (misalnya 20). Program kemudian harus menampilkan semua bilangan prima dari 1 sampai 20. Bilangan prima adalah bilangan bulat yang lebih besar dari 1 dan hanya dapat dibagi oleh dua angka, yaitu 1 dan angka itu sendiri.

3.4. Tugas Praktikum

Tugas praktikum dikerjakan dalam bentuk laporan praktikum. Implementasikan ke dalam kode C++ tiga soal di bawah ini. Implementasikan masing-masing soal berupa flowchart dan luaran untuk semua kasus/kondisi!

1. Buatlah sebuah program yang mencetak angka dari 1 sampai seterusnya (sesuai keinginan user), setiap angka dikeluarkan satu per baris, dengan aturan khusus sebagai berikut:
   A. Jika angka tersebut adalah kelipatan 3, cetak kata "Fizz" sebagai ganti angka.
   A. Jika angka tersebut adalah kelipatan 5, cetak kata "Buzz" sebagai ganti angka.
   A. Jika angka tersebut adalah kelipatan 3 dan 5 sekaligus, cetak kata "FizzBuzz".
2. Pak RT di komplek perumahan "Graha Koding" berencana untuk membagikan bingkisan spesial kepada rumah-rumah dengan nomor ganjil. Namun, karena komplek tersebut sangat besar, beliau meminta bantuan Anda, seorang programmer muda di komplek tersebut, untuk membuatkan program sederhana. Program ini harus bisa mendaftar semua nomor rumah ganjil, mulai dari nomor 2 hingga batas nomor rumah terakhir di komplek tersebut, agar Pak RT bisa dengan mudah mencetak daftarnya. Buatlah sebuah program yang pertama-tama akan menanyakan "Berapa jumlah total rumah di komplek?". Setelah pengguna memasukkan angka tersebut (misalnya 25), program Anda harus menggunakan perulangan `for` untuk memeriksa setiap nomor rumah dari 1 hingga 25 dan mencetak hanya nomor-nomor yang ganjil.
3. Seorang arsitek junior sedang merancang fasad (tampilan depan) sebuah gedung modern untuk merenovasi gedung A10 Unesa. Ia ingin membuat pola dekoratif berbentuk segitiga siku-siku menggunakan panel lampu. Untuk mempresentasikannya kepada stakeholder pihak Unesa, ia butuh program yang bisa mensimulasikan tampilan pola tersebut dalam berbagai ukuran. Pola ini terdiri dari blok-blok yang direpresentasikan dengan karakter '#'. Buatlah sebuah program yang meminta pengguna memasukkan sebuah angka yang merepresentasikan tinggi dari segitiga. Program kemudian harus mencetak pola segitiga siku-siku menggunakan karakter '#' sesuai dengan tinggi yang dimasukkan. Anda perlu menggunakan *nested for loop* (perulangan di dalam perulangan).

DAFTAR PUSTAKA

Stroustrup, B., 2024. *Programming: Principles and Practice Using C++*. Addison-Wesley Professional.

Stroustrup, B., 2013. *The C++ programming language*. Pearson Education.

Kirch-Prinz, U. and Prinz, P., 2002. *A complete guide to programming in C++*. Jones & Bartlett Learning.